Факультет компьютерных наук | Машинное обучение | Москва 2026

Лекция 8.2.
Метод k ближайших соседей (KNN)

Преподаватель: Меликян Алиса Валерьевна, amelikyan@hse.ru
кандидат наук, доцент Департамента программной инженерии ФКН НИУ ВШЭ,
академический руководитель магистерской программы
«Искусственный интеллект и продуктовый подход в HR-менеджменте»

Метод k ближайших соседей

k-Nearest Neighbors (kNN) – это простой и наглядный алгоритм машинного обучения, основанный на принципе того, что похожие объекты имеют похожие ответы.

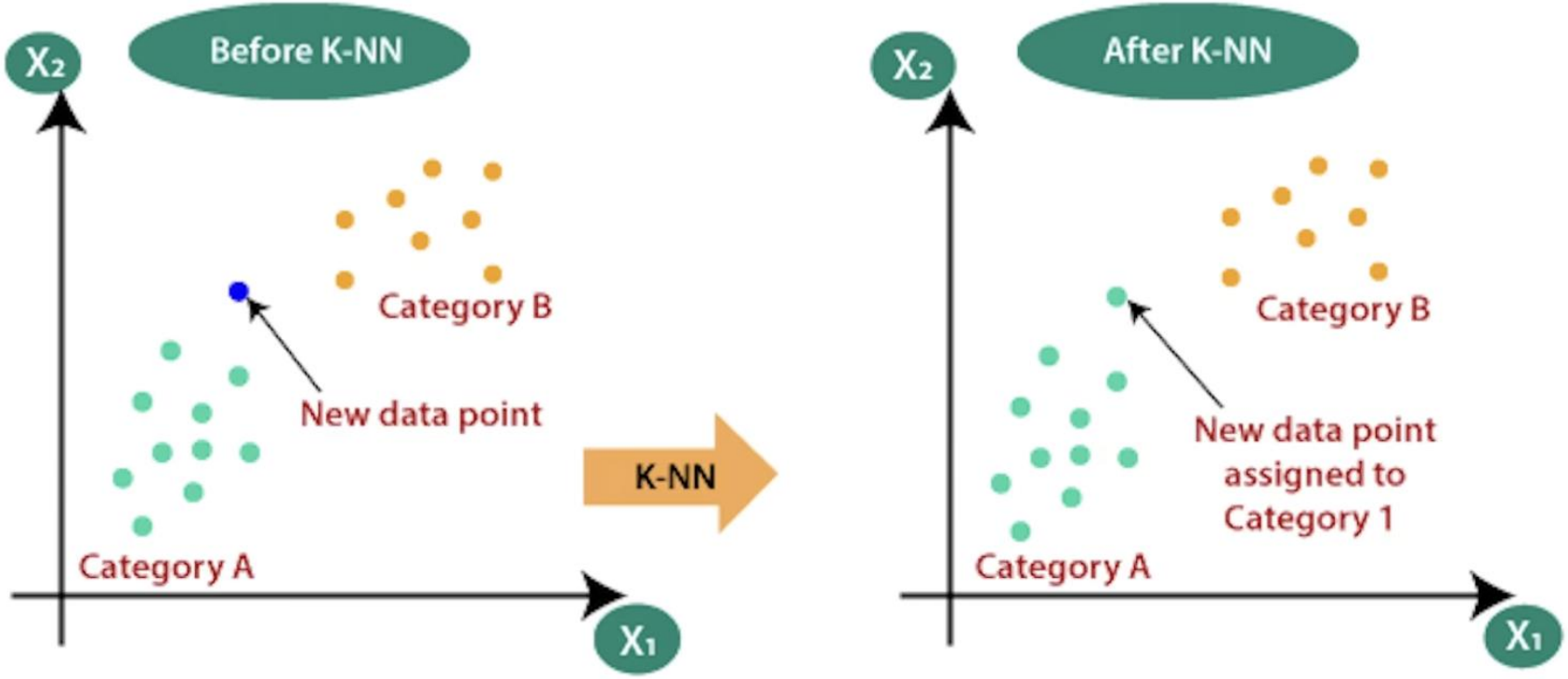
Для нового объекта находится k ближайших соседей в обучающей выборке и используются их значения для предсказания значения целевого признака для этого нового объекта.

Классификация → голосование

Регрессия → среднее

Основан на гипотезе компактности: *если два объекта похожи друг на друга, то ответы на них тоже похожи*.

Метод k ближайших соседей



Метод k ближайших соседей в HR

Задача: предсказать, уволится ли сотрудник

Признаки: возраст, стаж, зарплата, удовлетворённость

Идея алгоритма: если новый сотрудник похож на других сотрудников, то он, скорее всего, поведёт себя так же, как они.

Расчеты: для нового сотрудника считается расстояние от него до всех сотрудников и выбирается 5 самых близких сотрудников (ближайших соседей).

Предсказание: если большинство похожих сотрудников уволились, то предсказываем, что и новый сотрудник уволится (голосование большинства).

Алгоритм метода k ближайших соседей

Обучение состоит в запоминании обучающей выборки, конкретной формулы нет. В процессе работы алгоритма объекты в обучающей выборке сортируются по близости к новому объекту.

Дальше смотрим среди первых k ближайших соседей какой класс представлен больше всего. К этому классу мы и относим новый объект.

Гиперпараметры алгоритма – метрика расстояния и число соседей (k). Обычно значение k подбирается через кросс-валидацию.

Перед запуском алгоритма признаки нужно масштабировать, чтобы каждый признак вносил сопоставимый вклад в вычисление расстояния между объектами. Если признаки имеют разные масштабы, то признак с большими числовыми значениями будет доминировать в расстоянии.

Ленивый алгоритм (Lazy Learner)

kNN называют ленивым алгоритмом (Lazy Learner), потому что он не строит модель во время обучения, а хранит данные и выполняет все вычисления только при предсказании.

Что не делает kNN:

- не строит параметрическую модель;
- не оценивает коэффициенты;
- не минимизирует функцию потерь.

Что делает kNN:

- запоминает всю обучающую выборку;
- осуществляет вычисления во время предсказания.

Подбор числа соседей (k)

1. Выбрать функцию близости объектов.
2. Выбрать несколько возможных значений k, например: 1, 3, 5, 7, 9, 15, 25.
3. Для каждого значения рассчитать качество модели на валидации. Алгоритм не имеет собственной метрики качества – она зависит от задачи.

Задача классификации:

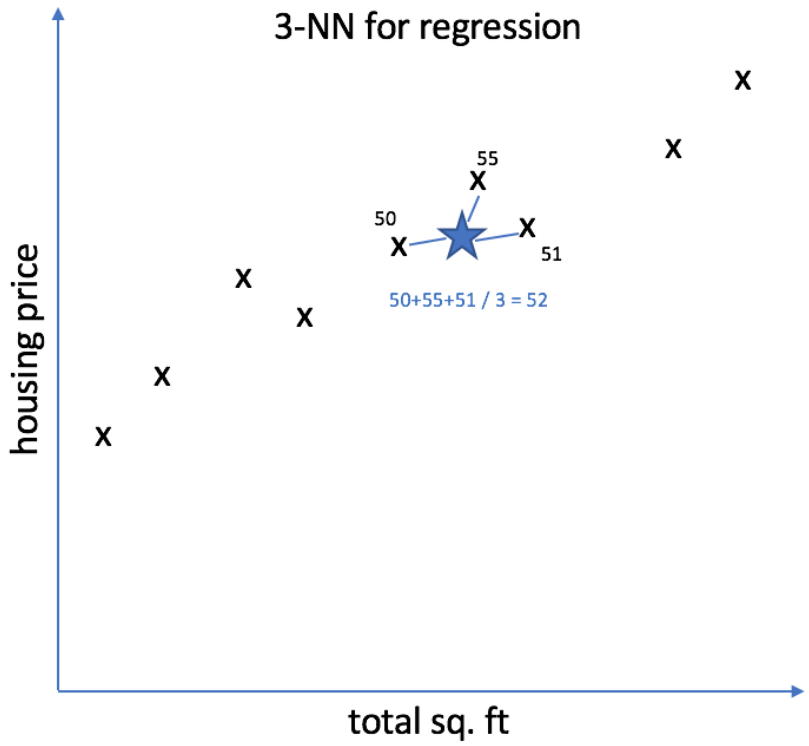
- Accuracy (при сбалансированных классах),
- Precision, Recall, F1-score (если классы не сбалансированы),
- ROC-AUC (если важен порог классификации).

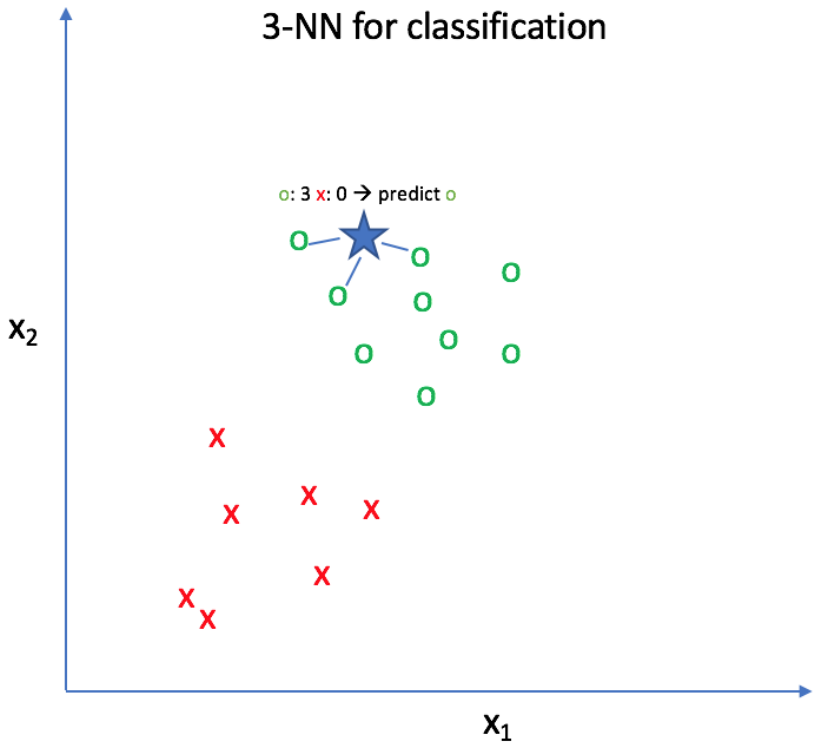
Задача регрессии:

- MSE (большие ошибки нужно штрафовать сильнее),
- RMSE (важна интерпретируемость в исходных единицах),
- MAE (есть выбросы, важна усредненная ошибка без усиленного штрафа).

GridSearchCV перебирает гиперпараметры, оценивает их через k-fold кросс-валидацию, выбирает лучшие и затем переобучает модель на всех данных с этими параметрами.

Классификация или регрессия



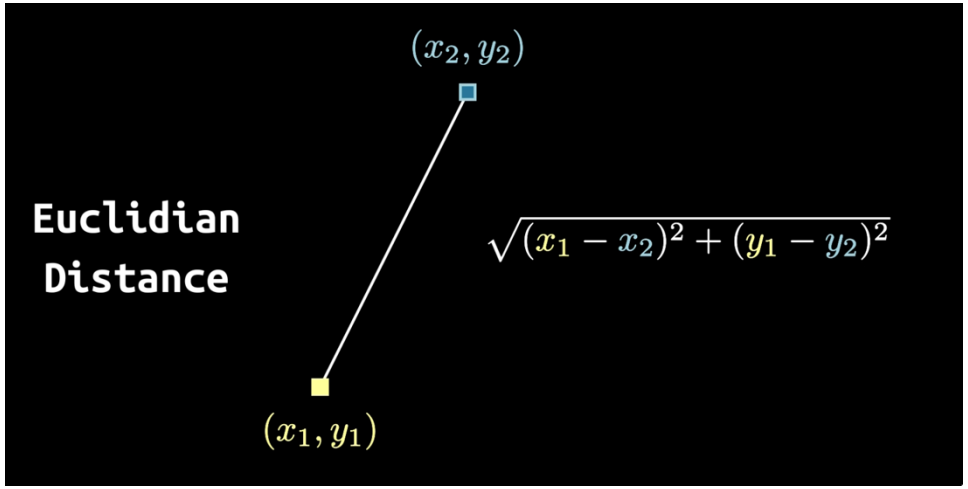


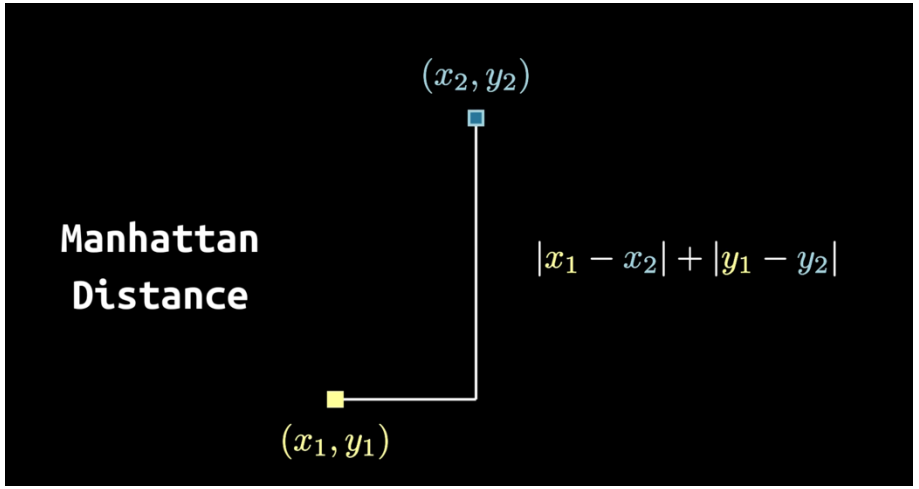
Функция близости объектов

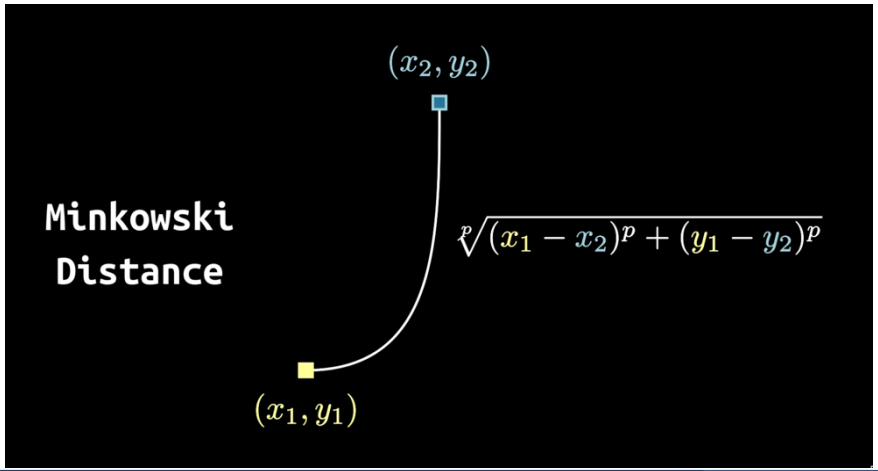
Функция близости – это метрика расстояния, которая определяет, насколько объекты похожи друг на друга. Алгоритм выбирает k объектов с минимальным расстоянием до точки запроса. Чем ближе объекты по анализируемым характеристикам, тем меньше число, измеряющее близость между ними.

- Евклидово расстояние (L2): подходит для непрерывных признаков;
- Манхэттенское расстояние (L1): более устойчиво к выбросам;
- Минковского расстояние: обобщение L1 и L2;
- Расстояние Хэмминга: для категориальных признаков;
- Расстояние Гауэра: для смешанных данных.

Функция близости объектов

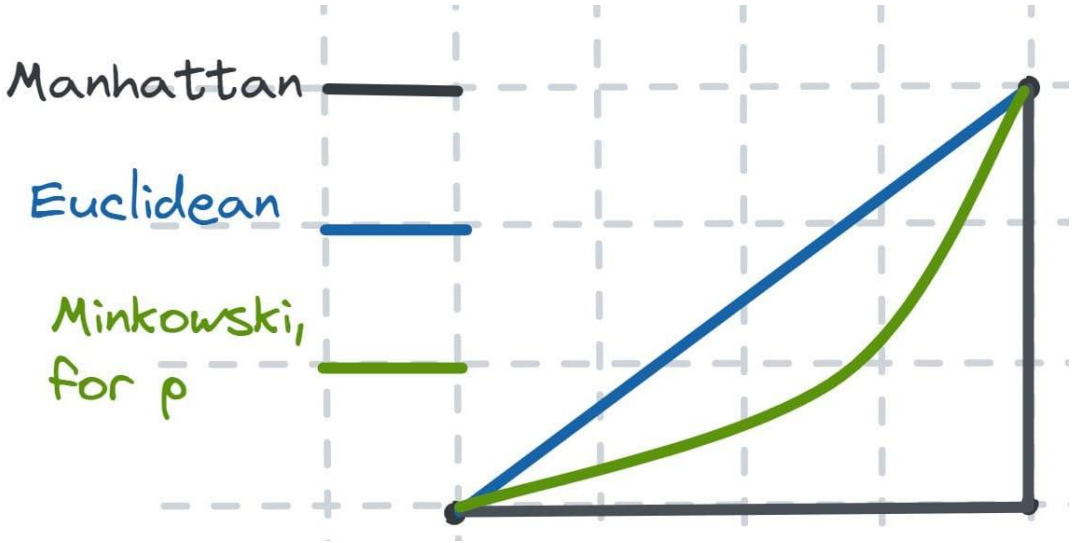






Расстояние Минковского

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$



p ≥ 1 – параметр (порядок метрики)

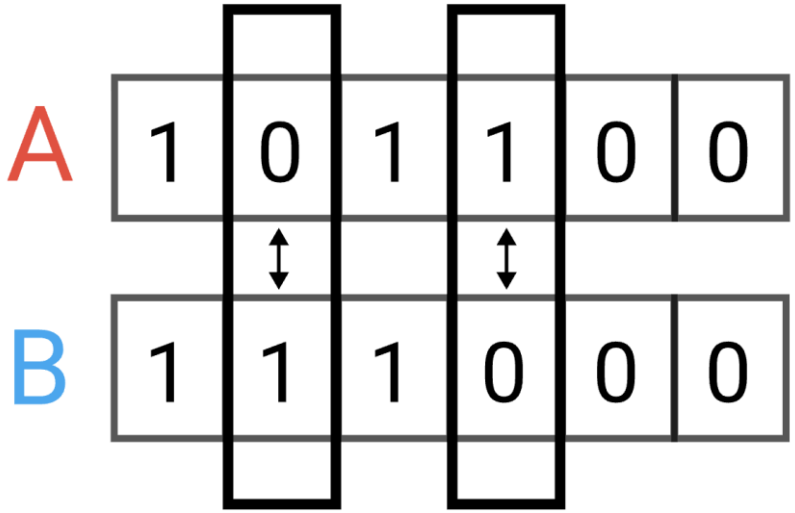
если p = 1 → Манхэттенское расстояние (радиус поиска – ромб)
если p = 2 → Евклидово расстояние (радиус поиска – круг)

Параметр p управляет тем, как сильно учитываются большие различия:

- большое p → соседство определяется по самому отличающемуся признаку
- маленькое p → учитывается совокупность всех различий

Расстояние Хэмминга

Расстояние Хэмминга – это мера различия между двумя векторами одинаковой длины, которая считает количество позиций, в которых они отличаются. Оно применяется к бинарным, строковым или категориальным данным: если элементы совпадают – вклад 0, если различаются – 1, после чего различия суммируются. Удобно использовать для признаков, где важен сам факт различия, а не его величина.



Расстояние Хэмминга

A	3	2	7	4	9	5	8	1
B	1	2	7	3	9	5	1	8

Чему равно расстояние Хэмминга?

Ответ: 4

A	“Geeks”	“for”	“Geeks”	“Java”	“C++”	“Swift”	“R”	“Python”
B	“Geeks”	“for”	“Geeks”	“C++”	“C”	“Swift”	“R”	“Python”

Ответ: 2

Расстояние Гауэра

Расстояние Гауэра (Gower distance) используется для измерения сходства объектов с разнородными признаками (числовыми и категориальными). Считается расстояние по каждому признаку отдельно, а потом рассчитывается среднее.

- для числовых: разница по модулю, деленная на размах;
- для категориальных (включая бинарные): совпадают → 0, различаются → 1.

В результате получается значение от 0 до 1, где 0 означает полное совпадение объектов, а 1 – максимальное различие. Это позволяет корректно находить ближайших соседей, не искажая расстояния из-за разных типов данных.

Расстояние Гауэра

$$d(x, y) = \frac{1}{n} \sum_{i=1}^{n} d_i$$

- **число:**

$$d_i = \frac{|x_i - y_i|}{\text{размах}}$$

- **категория / бинарный:**

$$d_i = \begin{cases} 0 & \text{если одинаковые} \\ 1 & \text{если разные} \end{cases}$$

Подбор числа соседей (k)

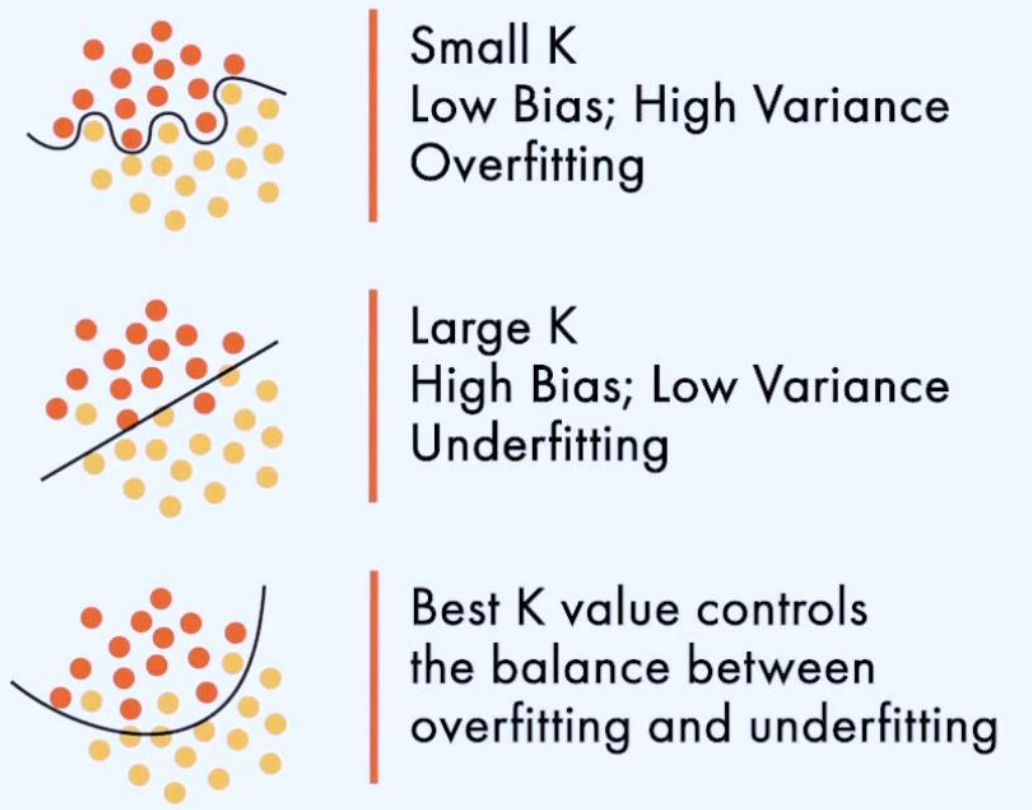
Важно подобрать оптимальное число соседей, потому что оно управляет балансом bias–variance:

Маленькое k:

- низкий bias,
- высокая variance,
- модель чувствительна к шуму,
- может переобучаться.

Большое k:

- выше bias,
- ниже variance,
- модель слишком усредняет,
- может недообучаться.



Подбор числа соседей (k)

Можно использовать в качестве стартовой точки начальное приближение:

$$k \approx \sqrt{n}$$

где n – число объектов в обучающей выборке.

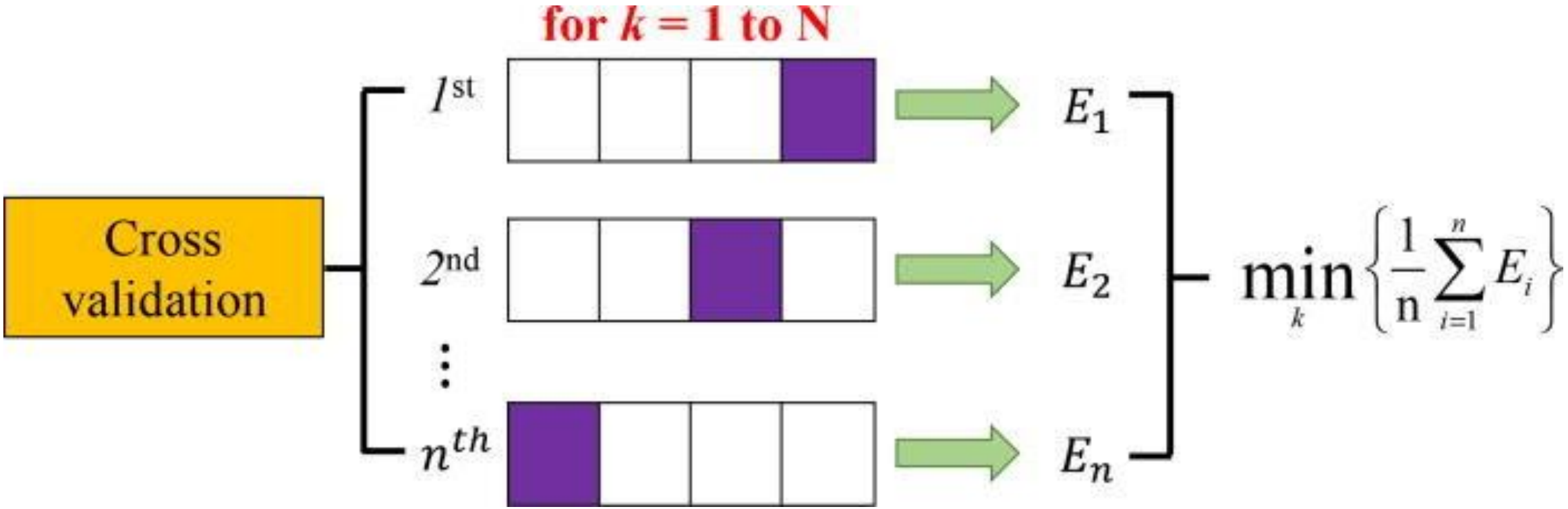
В классификации часто берут нечётное k, чтобы избежать ничьей.
При большом шуме лучше увеличивать k.
При малом количестве данных – снижать.

k определяет размер локального окружения:
маленький k → очень локальная модель
большой k → более глобальная

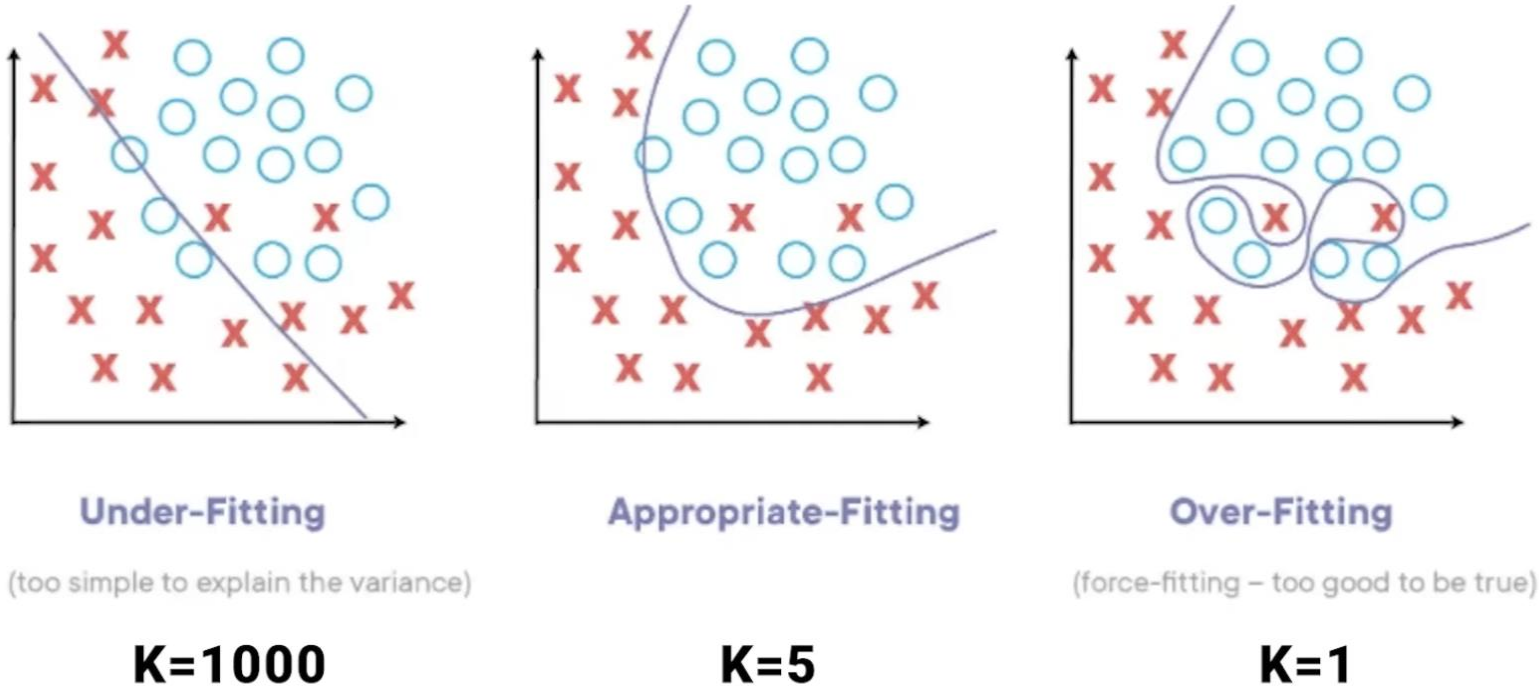
Подбор числа соседей через кросс-валидацию

1. Делим данные на k фолдов (например, 5)
2. Выбираем диапазон возможных значений для k
3. Для каждого k обучаем kNN на 4 фолдах, проверяем качество на оставшемся.
4. Повторяем 5 раз (каждый фолд побывает тестовым).
5. Получаем среднее качество для каждого k.
6. Выбираем лучшее k, это то, у которого:
   - наилучшее среднее качество,
   - стабильные результаты между фолдами.

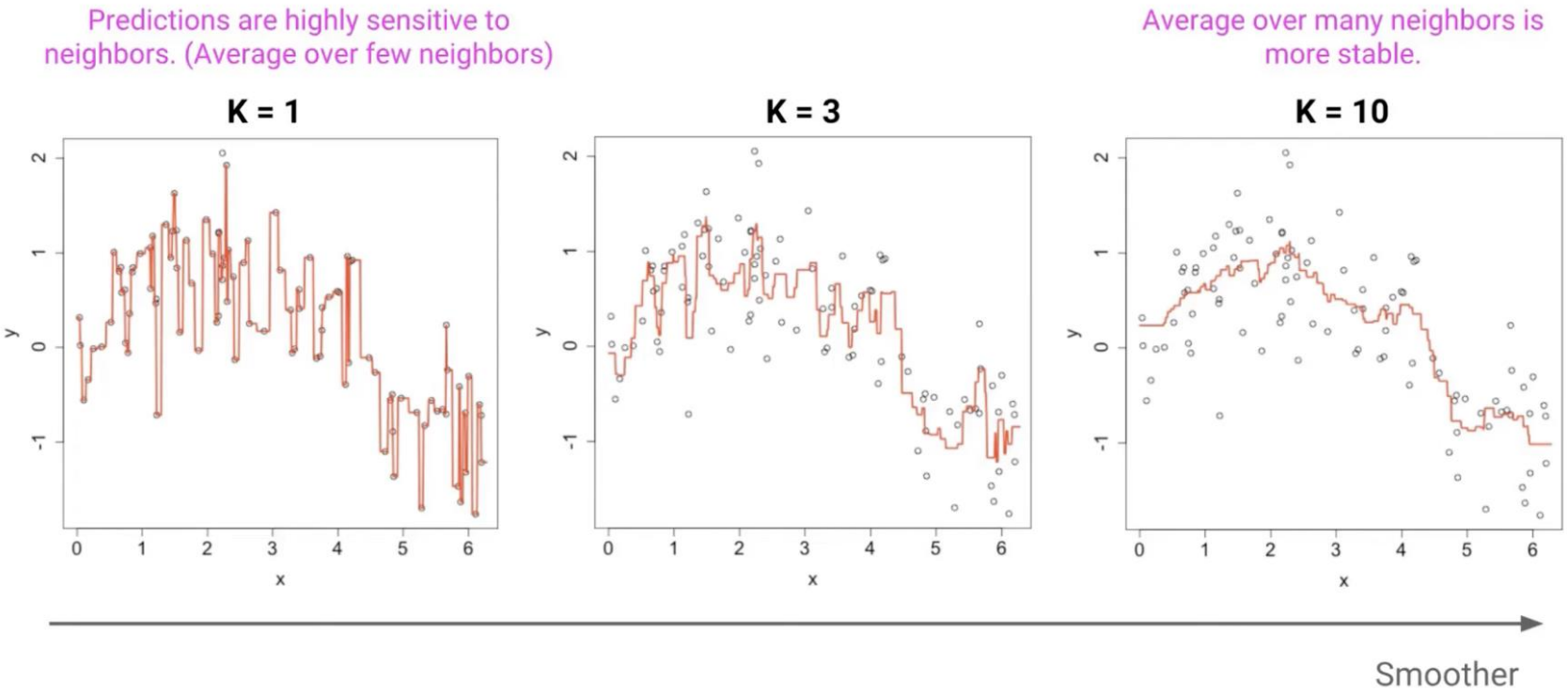
Подбор числа соседей через кросс-валидацию



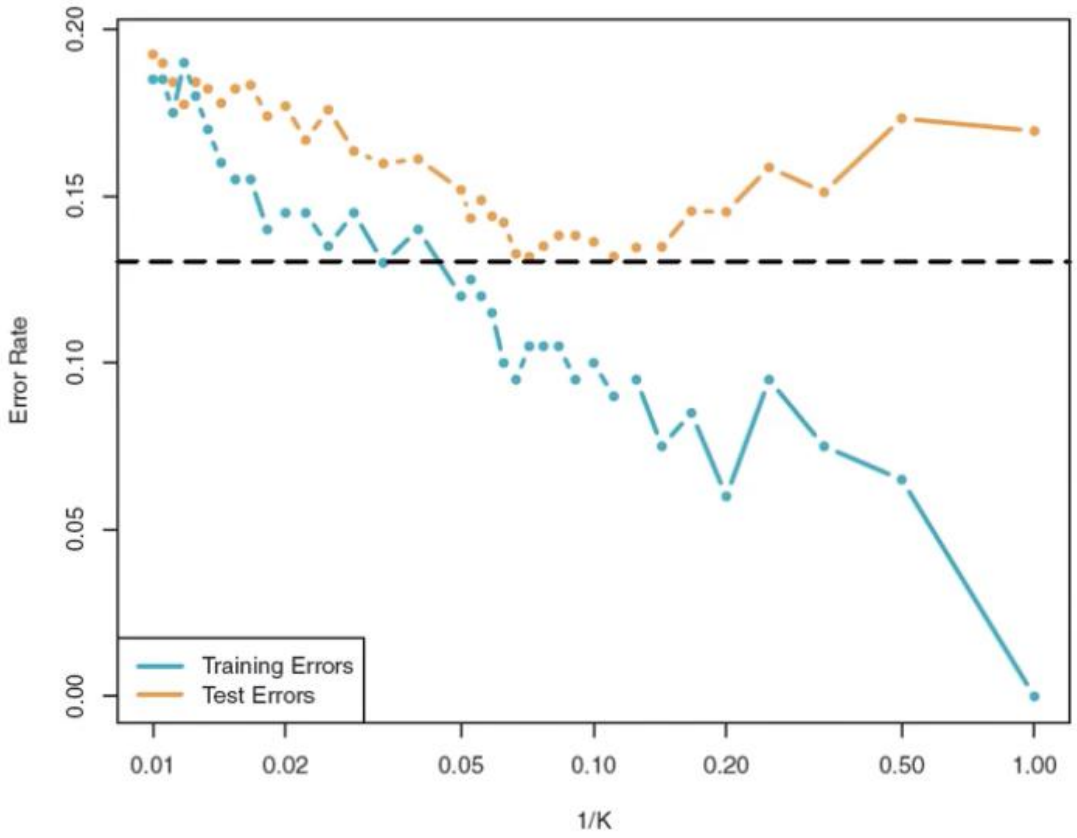
Подбор числа соседей (k)



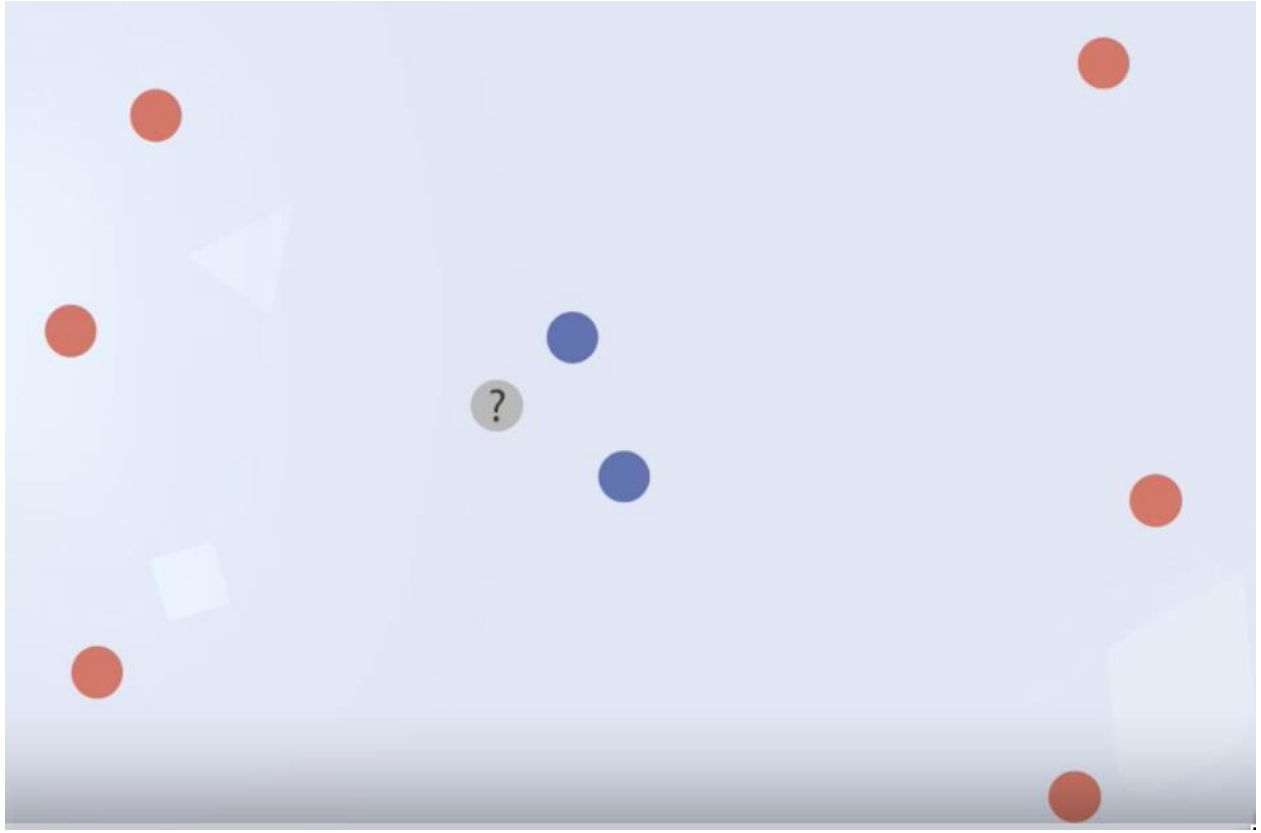
Подбор числа соседей (k)



Подбор числа соседей (k)



Вопросы



Если k = 3 к какому классу относится серая точка?

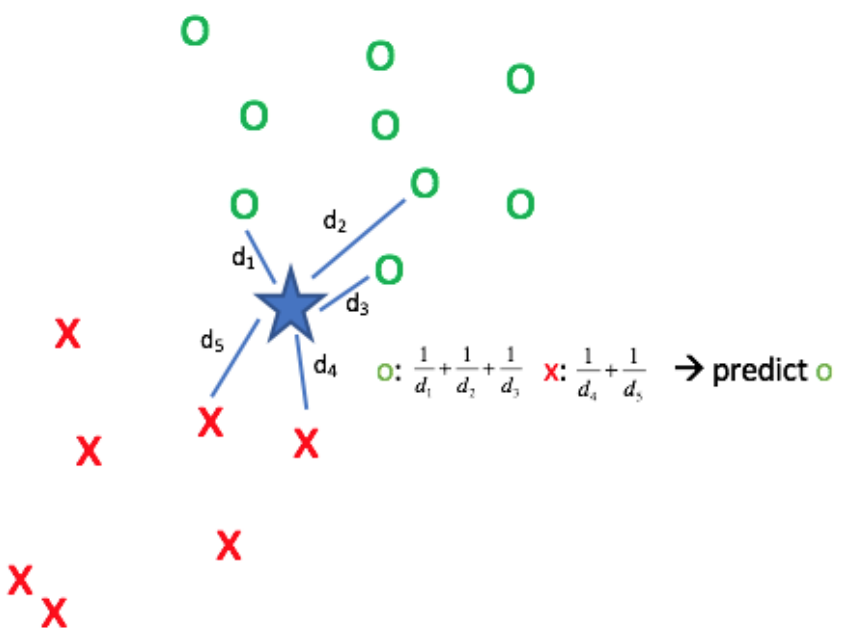
Если k = 5 к какому классу относится серая точка?

Если k = 8 к какому классу относится серая точка?

Взвешенный kNN

Взвешенный kNN – это модификация обычного kNN, в которой ближайшие соседи влияют на предсказание не одинаково, а с разными весами: чем ближе объект, тем больше его вклад. Это уменьшает влияние дальних и потенциально шумных точек.

В случае классификации это позволяет более точно определять класс, особенно при неоднородных распределениях данных, а в регрессии – получать взвешенное среднее значений соседей, что снижает влияние удалённых точек и уменьшает ошибку прогноза. Такой подход делает модель более гибкой и чувствительной к локальной структуре данных.



Взвешенный kNN

В регрессии предсказание – взвешенное среднее:

$$\hat{y} = \frac{\sum_{i=1}^{k} w_i \, y_i}{\sum_{i=1}^{k} w_i}$$

где w_i – вес соседа.

$$w_i = \frac{1}{d_i} \text{ или } w_i = \frac{1}{d_i^2}$$

В классификации каждый сосед голосует, но его голос умножается на вес:

$$\text{Score(class)} = \sum w_i$$

Побеждает класс с наибольшей суммой весов.

Преимущества kNN

- Простота понимания и реализации.
- Можно использовать разные метрики, что позволяет подстраивать алгоритм под тип данных.
- Мало гиперпараметоров (число соседей k и метрика расстояния).
- Хорошо работает на небольших наборах данных с четкой структурой.
- Подходит для многоклассовой классификации и регрессии.

Ограничения kNN

- Может медленно работать на больших данных.
- Не всегда легко подобрать корректное значение k.
- Чувствителен к шуму.
- Страдает от проклятия размерности: по мере увеличения количества признаков понятие расстояния становится менее значимым, затрудняя поиск соседей. Проблему можно решить снизив размерность данных.
- Важность признаков оценивается экспериментально, через влияние на точность модели при удалении или взвешивании признаков. Прямых коэффициентов, как в линейной регрессии, нет.

Использование kNN на практике

- Классификация – основной сценарий использования kNN.
- Регрессия – возможна, но применяется реже, обычно на небольших или средних данных.
- Рекомендательные системы (находит объекты, похожие на интересующие пользователя).
- Поиск аномалий (выявление необычных объектов, сильно отличающихся от других).
- Импутация пропущенных данных.

Спасибо за внимание!